

Hough Transform

Algorithm Description

The Hough transform detects lines by converting the line detection problem into a peak finding problem in a parameter space. A point (x, y) in image space satisfying $\rho = x \cdot \cos(\theta) + y \cdot \sin(\theta)$ traces a sinusoidal curve in the ρ - θ parameter space. When multiple collinear points in image space each cast their sinusoid into the accumulator, those sinusoids intersect at a common (ρ, θ) coordinate representing the line they share. Finding lines reduces to finding where the most sinusoids intersect, which is where the accumulator has the highest vote counts.

I first ran a Canny edge detector on each image to produce a binary edge map. I allocated a two-dimensional accumulator array with rows indexed by ρ and columns indexed by θ . I defined θ over $[-\pi/2, \pi/2)$ to handle all line orientations, including vertical lines, which would require $m = \infty$ in slope-intercept form. I computed the maximum diagonal distance $D = \sqrt{(\text{rows}^2 + \text{cols}^2)}$ and restricted ρ to $[-D, D]$. For each edge pixel, I computed ρ for every θ bin using $\rho = x \cdot \cos(\theta) + y \cdot \sin(\theta)$ and incremented the corresponding accumulator cell.

To detect significant intersections, I applied a local maximum filter over a neighborhood window and required each candidate peak to also exceed a global threshold set as a fixed ratio of the accumulator maximum. I then sorted surviving candidates by vote count and discarded any peak whose (ρ, θ) index fell within the neighborhood distance of a higher-ranked peak already accepted, preventing the same physical edge from producing multiple adjacent detections. The accepted peaks were converted back to Cartesian line parameters and drawn over the original image.

Results Analysis

Figure 1 shows the full pipeline on all three images. The parameter space column shows the sinusoidal curves produced by each edge pixel's votes, and the bright intersection points visible in those plots correspond directly to the dominant lines in the scene. The significant intersection column confirms that the peak detection isolates exactly those bright points. For the synthetic test images, the detected lines track the shape boundaries cleanly. For the real-world input photograph, raising the peak ratio to 0.65 was necessary to reject the hand and background texture and keep only the four dominant paper edges.

Figure 2 documents the effect of varying K_θ across 45, 90, 180, and 360 bins. At $K_\theta=45$, the angular resolution is coarse enough that each physical edge gets assigned to the nearest available bin rather than its true angle, causing the drawn lines to visibly deviate from the actual boundaries. At $K_\theta=180$, the resolution is fine enough that lines align correctly with the shape edges. $K_\theta=360$ produces nearly identical results to 180 for these image sizes, because the added angular precision is smaller than the localization error introduced by the edge detector itself. I scaled the neighborhood suppression radius proportionally with K_θ so that a finer grid does not cause a single physical edge to register as multiple adjacent peaks.

Figure 3 isolates the peak detection stage directly. The left column shows the full accumulator, the center column shows only the thresholded local maxima, and the right column shows the resulting lines. The number of peaks in the center column matches the number of edges in each shape, confirming that the two-stage detection of local maximum filter followed by global threshold and duplicate suppression correctly identifies one intersection per physical line.